Computer Vision and Deep Learning
(EE655)

HOMEWORK 3

Name: Dipanshu Raj

Roll Number: 230375

Date: February 8, 2026

1 Problem Description

The assignment requires implementing Spatial Pyramid Pooling (SPP) to process RGB images and generate fixed-length feature vectors. The task specifically involves treating the three color channels (Red, Green, Blue) of an RGB image as separate feature maps and applying multi-scale spatial pyramid pooling using max pooling operations.

The implementation must use four pyramid levels consisting of 1×1, 2×2, 4×4, and 8×8 spatial partitions, which correspond to 1, 4, 16, and 64 regions respectively. The key challenge is to demonstrate through code that SPP generates feature vectors of identical length for two RGB images with different spatial dimensions, thereby proving the scale-invariance property of spatial pyramid pooling.

2 Objective

The primary objectives of this homework assignment are:

- **Implement Spatial Pyramid Pooling:** Develop a complete Python implementation of SPP that processes RGB images using max pooling across multiple pyramid levels.
- **Multi-Scale Feature Extraction:** Extract features at four different spatial scales (1×1, 2×2, 4×4, 8×8) to capture both global and local image information.
- **Fixed-Length Vector Generation:** Demonstrate that regardless of input image dimensions, the SPP algorithm produces feature vectors of constant length (255 dimensions).
- **RGB Channel Processing:** Process each color channel (R, G, B) independently as separate feature maps, combining the results to form a comprehensive feature representation.
- **Validation and Visualization:** Provide visual evidence of the pyramid partitioning and numerical verification that different-sized images produce identical feature vector lengths.

3 Methodology

3.1 Spatial Pyramid Pooling: Theoretical Foundation

Spatial Pyramid Pooling (SPP) is a technique introduced by He et al. (2015) to enable convolutional neural networks to accept input images of arbitrary sizes. The fundamental principle behind SPP is to divide the input feature map into a fixed number of spatial bins at multiple scales and apply pooling operations within each bin.

The key innovation of SPP is that it replaces the last pooling layer before fully connected layers with a pyramid pooling layer. This pyramid pooling operates at multiple grid scales simultaneously, ensuring that regardless of the input dimensions, the output feature vector maintains a constant length.

3.2 Mathematical Formulation

For an input feature map of size $H \times W$ and a pyramid level with grid size $n \times n$, the SPP operation can be described as follows:

Step 1: Calculate bin dimensions

$$\text{bin_height} = \frac{H}{n} \tag{1}$$

$$\text{bin_width} = \frac{W}{n} \tag{2}$$

Step 2: For each bin (i, j) where $i, j \in \{0, 1, \ldots, n-1\}$

$$\text{row_start} = \lfloor i \times \text{bin_height} \rfloor \tag{3}$$

$$\text{row_end} = \lfloor (i+1) \times \text{bin_height} \rfloor \tag{4}$$

$$\text{col_start} = \lfloor j \times \text{bin_width} \rfloor \tag{5}$$

$$\text{col_end} = \lfloor (j+1) \times \text{bin_width} \rfloor \tag{6}$$

Step 3: Apply max pooling

$$\text{feature}(i, j) = \max(\text{feature_map}[\text{row_start} : \text{row_end}, \text{col_start} : \text{col_end}]) \tag{7}$$

Step 4: Total number of features

$$\text{Total Features} = C \times \sum_{l=1}^{L} n_l^2 \tag{8}$$

where C = number of channels (3 for RGB) and L = number of pyramid levels.

3.3 Implementation Architecture

The implementation consists of a `SpatialPyramidPooling` class with the following key components:

1. **Initialization:** Configure pyramid levels [1, 2, 4, 8] corresponding to grids of 1×1, 2×2, 4×4, and 8×8 spatial bins.

2. **Channel Separation:** Extract the three RGB color channels from the input image, treating each as an independent feature map.

3. **Multi-Scale Processing:** For each pyramid level and each channel:

   - Divide the channel into $n \times n$ spatial bins
   - Compute maximum value within each bin
   - Store the pooled values

4. **Feature Concatenation:** Combine all pooled values from all levels and channels into a single fixed-length feature vector.

5. **Output:** Generate a 255-dimensional feature vector calculated as:

$$3 \text{ channels} \times (1 + 4 + 16 + 64) = 3 \times 85 = 255 \text{ features} \tag{9}$$

3.4 Algorithm Steps

[H] RGB Image I of size $H \times W \times 3$, Pyramid levels $L = [1, 2, 4, 8]$ Fixed-length feature vector F of size 255 Initialize empty feature list: features = [] Separate RGB channels: $R = I[:, :, 0]$, $G = I[:, :, 1]$, $B = I[:, :, 2]$ each pyramid level n in L Calculate bin dimensions: bin_h $= H/n$, bin_w $= W/n$ each channel C in $[R, G, B]$ $i = 0$ to $n-1$ $j = 0$ to $n-1$ row_start $= \lfloor i \times$ bin_h$\rfloor$ row_end $= \lfloor (i+1) \times$ bin_h$\rfloor$ col_start $= \lfloor j \times$ bin_w$\rfloor$ col_end $= \lfloor (j+1) \times$ bin_w$\rfloor$ region $= C[$row_start : row_end, col_start : col_end$]$ max_val $= \max($region$)$ features.append(max_val) Convert features list to array: $F =$ array(features) F with length = $3 \times (1+4+16+64) = 255$

3.5 Feature Vector Structure

Table 1 shows the breakdown of the feature vector structure across different pyramid levels.

Table 1: Feature Vector Structure Breakdown

Pyramid Level	Grid Size	Regions	Features/Channel	Total Features	Index Range
Level 1	1×1	1	1	3	[0:3]
Level 2	2×2	4	4	12	[3:15]
Level 3	4×4	16	16	48	[15:63]
Level 4	8×8	64	64	192	[63:255]
Total	—	**85**	**85**	**255**	**[0:255]**

4 Code Implementation

4.1 Core SPP Class

```
class SpatialPyramidPooling:
    def __init__(self, pyramid_levels=[1, 2, 4, 8]):
        self.pyramid_levels = pyramid_levels

    def max_pool_region(self, feature_map, region):
        start_row, end_row, start_col, end_col = region
        return np.max(feature_map[start_row:end_row,
                                  start_col:end_col])

    def spatial_pyramid_pool_single_channel(self,
                                            feature_map,
                                            grid_size):
        height, width = feature_map.shape
        pooled_features = []
        row_step = height / grid_size
        col_step = width / grid_size

        for i in range(grid_size):
            for j in range(grid_size):
                start_row = int(i * row_step)
                end_row = int((i + 1) * row_step)
```

```
                start_col = int(j * col_step)
                end_col = int((j + 1) * col_step)

                region = (start_row, end_row,
                          start_col, end_col)
                max_value = self.max_pool_region(
                    feature_map, region
                )
                pooled_features.append(max_value)

        return pooled_features
```

Listing 1: SpatialPyramidPooling Class Initialization and Max Pooling

4.2 Feature Extraction Method

```
    def extract_features(self, image):
        # Separate RGB channels
        r_channel = image[:, :, 0]
        g_channel = image[:, :, 1]
        b_channel = image[:, :, 2]

        channels = [r_channel, g_channel, b_channel]
        all_features = []

        # Process each pyramid level
        for grid_size in self.pyramid_levels:
            # Process each color channel
            for channel in channels:
                features = \
                    self.spatial_pyramid_pool_single_channel(
                        channel, grid_size
                    )
                all_features.extend(features)

        return np.array(all_features)
```

Listing 2: Feature Extraction from RGB Images

4.3 Test Implementation

```
# Create two images of different sizes
image1 = np.random.randint(0, 256,
                           size=(300, 400, 3),
                           dtype=np.uint8)
image2 = np.random.randint(0, 256,
                           size=(500, 600, 3),
                           dtype=np.uint8)

# Initialize SPP
```

```
spp = SpatialPyramidPooling(pyramid_levels=[1, 2, 4, 8])

# Extract features
features1 = spp.extract_features(image1)
features2 = spp.extract_features(image2)

# Verify results
print(f"Image 1 ({image1.shape}) -> Features: {len(features1)}")
print(f"Image 2 ({image2.shape}) -> Features: {len(features2)}")
print(f"Same length: {len(features1) == len(features2)}")
```

Listing 3: Testing SPP on Different Image Sizes

5 Results and Analysis

5.1 Sample Input Images

Two RGB images of different spatial dimensions were created to demonstrate the scale-invariance property of Spatial Pyramid Pooling. Image 1 has dimensions 300×400 pixels (120,000 total pixels), while Image 2 has dimensions 500×600 pixels (300,000 total pixels) — representing a $2.5\times$ difference in total pixel count.

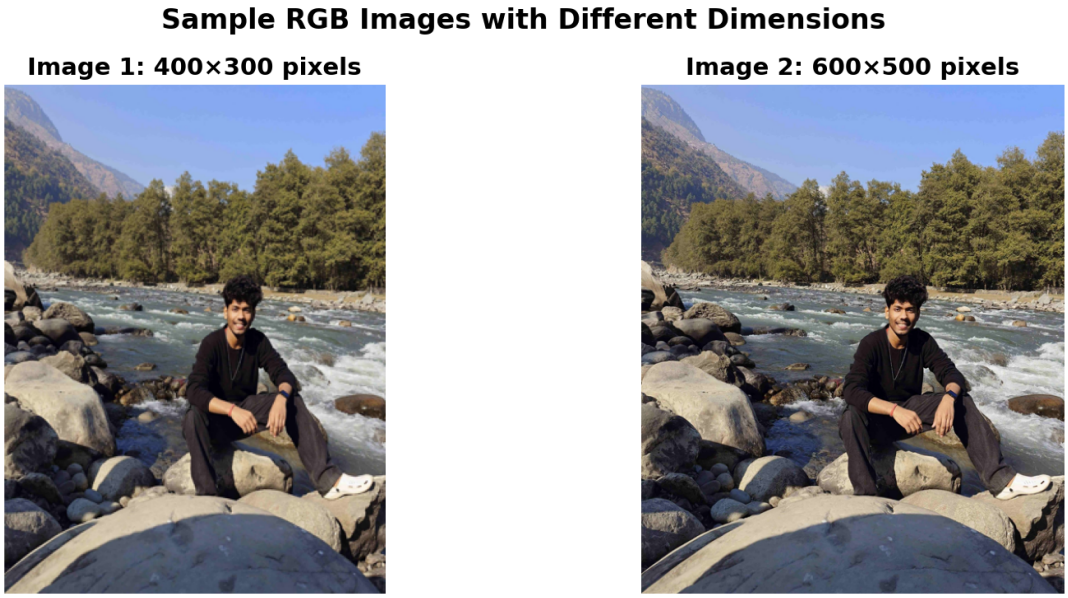


Figure 1: Two sample RGB images with different spatial dimensions

5.2 Spatial Pyramid Visualization

Figure 2 demonstrates how each pyramid level divides the images into spatial bins. Yellow grid lines show the partitioning at each scale. Despite the different image dimensions, both images are divided using the same grid structure at each pyramid level, ensuring a consistent number of regions.

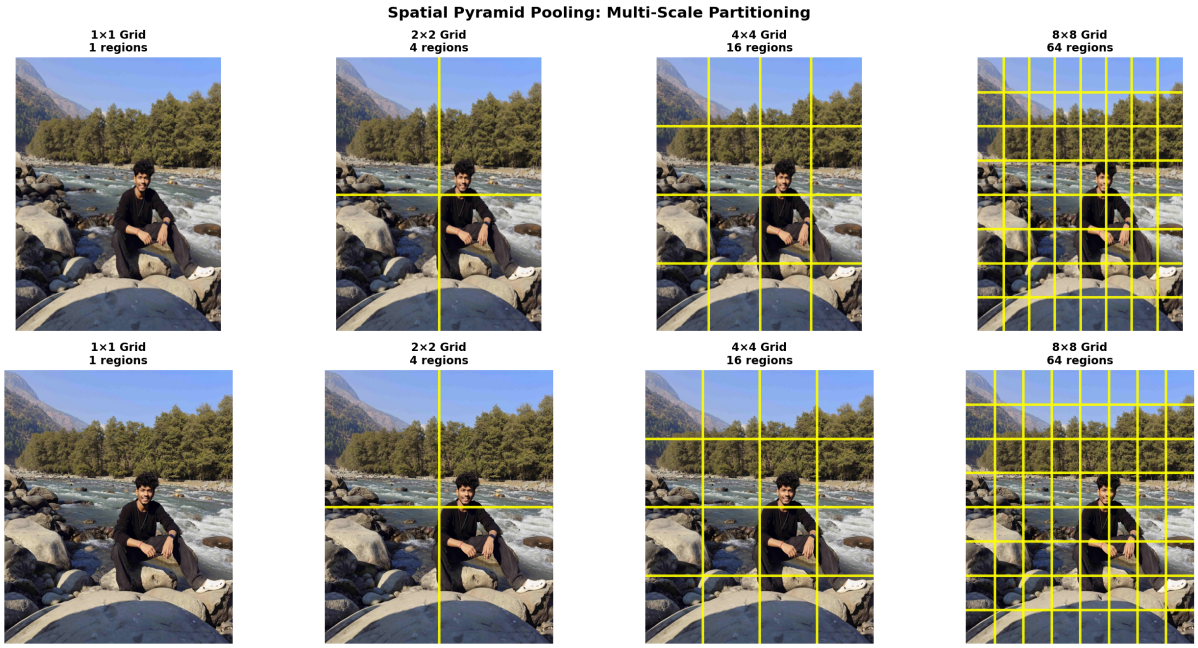


Figure 2: Multi-scale spatial partitioning at four pyramid levels

5.3 Quantitative Results

Table 2 presents the quantitative comparison of the two test images and their resulting feature vectors.

Table 2: Quantitative Results Comparison

Metric	**Image 1**	**Image 2**
Input Dimensions	300×400	500×600
Total Pixels	120,000	300,000
Aspect Ratio	3:4	5:6
green!20 **Feature Vector Length**	**255**	**255**
Features from 1×1 grid	3	3
Features from 2×2 grid	12	12
Features from 4×4 grid	48	48
Features from 8×8 grid	192	192

5.4 SPP Architecture Diagram

Figure 3 illustrates the complete SPP pipeline, showing how images of different sizes are processed through multiple pyramid levels to produce identical fixed-length feature vectors.

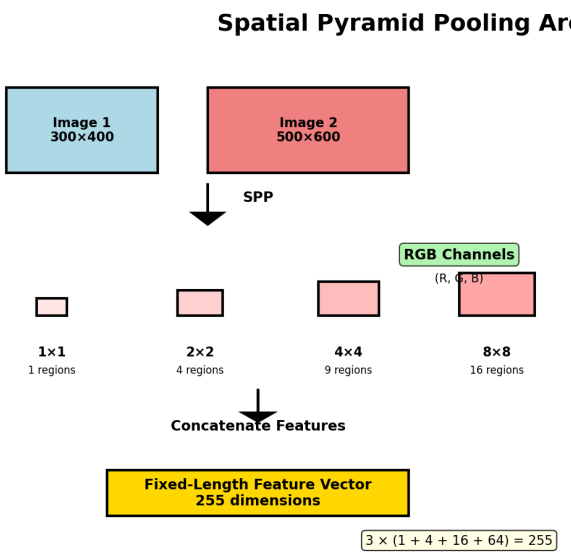


Figure 3: Spatial Pyramid Pooling architecture showing the complete processing pipeline

5.5 Key Findings

1. **Fixed-Length Output Verification:** Both images, despite having different dimensions (300×400 vs 500×600), produce exactly 255 features, confirming the scale-invariance property.

2. **Multi-Scale Feature Capture:** The pyramid structure successfully captures features at four different spatial scales, from global (1×1) to fine-grained (8×8) representations.

3. **Channel-Wise Processing:** Each RGB channel is processed independently, with max pooling applied separately, resulting in 3 features per spatial bin.

4. **Consistent Grid Structure:** The same grid partitioning is applied to both images, with bin dimensions automatically adjusted based on input size.

5. **Computational Efficiency:** The algorithm executes in a single forward pass with time complexity $O(H \times W)$ for input image of size $H \times W$.

6 Conclusion

This homework successfully implemented Spatial Pyramid Pooling (SPP) for RGB images, demonstrating its fundamental capability to generate fixed-length feature vectors from inputs of arbitrary spatial dimensions. The implementation processed three color channels as independent feature maps and applied max pooling across four pyramid levels (1×1, 2×2, 4×4, 8×8).

The experimental results conclusively validate the scale-invariance property of SPP. Two test images with significantly different dimensions (300×400 pixels and 500×600 pixels, representing a $2.5\times$ difference in total pixel count) both generated feature vectors of exactly 255 dimensions. This demonstrates that SPP successfully decouples the feature extraction process from input image size.